

LeetCode 1905: Count Sub Islands

Authored by: Arpit Raj, LNMIIT Jaipur | Date: August 4, 2026

Logic & Approach Notes

- dsu approach se krenge to
- pehle grid2 me dsu krke sab ko merge kr lenge
- phir total island nikaal lenge grid 2 me
- phir invalid island nikaal lenge
- i.e jo grid2 me 1 he lekin grid 1 me 0 he
- return total-invalid=valid

Disjoint Set Union (DSU) Structure

```
class dsu{
public:
    vector<int> parent;
    vector<int> size;
    dsu(int n){
        parent.resize(n);
        size.resize(n,1);
        for(int i=0;i<n;i++){
            parent[i]=i;
        }
    }

    int findpar(int node){
        if(parent[node]==node){
            return node;
        }
        parent[node]=findpar(parent[node]);
        return parent[node];
    }

    void merge(int u,int v){
        int pu=findpar(u);
        int pv=findpar(v);
        if(pu==pv) return;
        if(size[pu]<size[pv]) swap(pu,pv);
        size[pu]+=size[pv];
        parent[pv]=pu;
    }
};
```

| Main Solution

```

class Solution {
public:
    int countSubIslands(vector<vector<int>>& grid1, vector<vector<int>>& grid2) {

        //dsu approach se krenge to
        //pehle grid2 me dsu krke sab ko merge kr lenge
        //phir total island nikaal lenge grid 2 me
        //phir invalid island nikaal lenge
        //i.e jo grid2 me 1 he lekin grid 1 me 0 he
        //return total-invalid=valid

        int n=grid1.size();
        int m=grid1[0].size();

        dsu ds(n*m);

        vector<pair<int,int>> dir={{-1,0},{0,-1},{0,1},{1,0}};

        for(int r=0;r<n;r++){
            for(int c=0;c<m;c++){
                if(grid2[r][c]==0) continue;
                int curr=r*m+c;
                for(auto &it:dir){
                    int nr=r+it.first;
                    int nc=c+it.second;
                    int next=nr*m + nc;
                    if(nr>=0 && nc>=0 && nr<n && nc<m){
                        if(grid2[nr][nc]==0) continue;
                        ds.merge(curr,next);
                    }
                }
            }
        }

        int tot=0;
        int inval=0;
        unordered_set<int> all;
        unordered_set<int> inv;
        for(int r=0;r<n;r++){
            for(int c=0;c<m;c++){
                int curr=r*m+c;
                if(grid2[r][c]==0) continue;
                all.insert(ds.findpar(curr));
            }
        }

        tot=all.size();

        for(int r=0;r<n;r++){
            for(int c=0;c<m;c++){
                int curr=r*m+c;
                if(grid2[r][c]==0) continue;

                if(grid2[r][c]==1 && grid1[r][c]==0)
                    inv.insert(ds.findpar(curr));
            }
        }

        inval=inv.size();
    }
};

```

```
        return tot-inval;  
    }  
};
```